

Curso:

FUNDAMENTOS DE LA PROGRAMACIÓN

Módulo 1:

PROGRAMACIÓN ESTRUCTURADA

Unidad 2:

PSEUDOCÓDIGO



Presentación

En esta Unidad nos enfocaremos en la forma de representar algoritmos que usaremos en este y el próximo Módulo: el pseudocódigo.



Objetivos

Que los participantes ...

- Conozcan e incorporen la forma de representación de algoritmos que serán utilizados en este Módulo: el pseudocódigo
- Conozcan e incorporen las convenciones de los mismos
- Conozcan e incorporen los aspectos principales de la representación de algoritmos usando pseudocódigo



Bloques temáticos

1. Pseudocódigo
2. Ejemplo integrador
3. Tips sobre el desarrollo en pseudocódigo

ANEXO 1 - Respuesta Actividad de Análisis

ANEXO 2 - Checklist de código

ANEXO 3 - Preguntas Frecuentes

1. Pseudocódigo

El pseudocódigo, cuyo significado literal es “falso lenguaje”, es el tipo de descripción de alto nivel para representar algoritmos, sobre el que se encuentra basado el presente Curso. De allí la importancia que le daremos a este tópico en la presente Unidad y en las siguientes. Esta importancia hace necesario definir el uso que le daremos al pseudocódigo, ya que, al no haber un estándar unilateralmente aceptado, es posible encontrar numerosas formas de representarlo. Por lo que, a continuación, daremos algunos parámetros (que iremos enriqueciendo) para formalizar su uso en este Curso.

Como nombramos anteriormente, el **pseudocódigo** es un subconjunto del lenguaje natural con algunas estructuras sintácticas propias que lo asemejan a los lenguajes de programación, como ser asignaciones, ciclos, condicionales y otros tipos de estructuras. Si bien su uso actual suele estar limitado al ámbito académico como base ~~para~~ teórico-práctica que permita una rápida adaptación a lenguajes de programación concretos, también suele estar presente en documentos de investigación y divulgación en los que resulta importante **abstraerse de una implementación (lenguaje) en particular**. También suele ser de gran ayuda en varias instancias en proyectos de desarrollo de software, por ejemplo, cuando **para** surge la necesidad de **validar con un usuario** o actor **no técnico un algoritmo**, permitiendo mostrarlo en forma genérica **sin la necesidad de que aquella persona conozca** el lenguaje **de programación** en el cual finalmente será implementado.

En algunos casos estos se conoce como "meta lenguaje" al utilizar palabras y formulaciones propias del "negocio" o del ámbito en que se encuentren las personas que requieran validar una solución o comprender un problema en forma previa a la programación.

Para que este proceso de construcción sea efectivo, resulta necesario presentar un **pseudocódigo ordenado, limpio y claro**, con el fin de que sea totalmente

comprensible en forma inmediata. Por ello, nos vemos obligados a seguir algunas reglas o “mejores prácticas” que analizaremos a continuación.

1.1 Convenciones

Nos centraremos en definir los elementos que utilizaremos en nuestros primeros pseudocódigos y posteriormente daremos algunos parámetros generales que deberán cumplir la estructura de todos los pseudocódigos que realicemos.

Definiremos algunas “palabras reservadas” que tendrán una función específica. Inicialmente nos concentraremos en los que vamos a utilizar en esta Unidad, para ir incorporando el resto en el transcurso del Curso.

Por “palabra reservada” nos referimos a todos aquellos términos especiales que utilizaremos para definir las instrucciones que son propias del lenguaje, en este caso, del pseudocódigo.

Pseudocódigo	
Palabra reservada	Descripción
"programa NOMBRE"	Nos indica que comienzo comenzó del programa. Siempre va acompañado del nombre del programa, que debe ser representativo de lo que hace
"inicio" / "fin"	Bloque principal donde vamos a ubicar nuestro código. Solo hay 1 "inicio" y solo hay 1 "fin" en todos los programas. El "inicio" siempre va luego del "programa..." y el "fin" es siempre la última línea de todo programa
"mostrar:"	Representa una salida en formato de texto del programa. Se pueden imprimir mensajes (siempre entre '"') o valores de variable (tema que comentaremos un poco más adelante) La sintaxis es “mostrar:” + salida de datos e información. En el caso de querer mostrar un mensaje, este deberá estar entre comillas dobles, por ejemplo:

mostrar: "el resultado es:" → muestra texto
mostrar: valorDelCalculo → muestra variable

"ingresar:"

Sirve para simular la acción en la cual un supuesto usuario de nuestro programa (la persona ~~+~~ que lo "ejecutaría") ~~ingresar~~ un valor por teclado y ese valor queda guardado en una variable

La sintaxis sería "ingresar:" + entrada de datos

El caso de querer guardar un nombre de una persona en una variable sería, por ejemplo:

ingresar: nombreDeLaPersona

Siempre escribiremos en minúscula, salvo para un caso puntual que definiremos más adelante.

En caso de tener nombres de programas o instrucciones que involucren más de una palabra (nombre compuesto), utilizaremos un patrón de escritura conocido como "lower camel case". Este es un derivado del término "camel case" en inglés y que define que una sentencia (más de una palabra) se unifique en una palabra compuesta en la que se unen todas las palabras de la sentencia, aunque poniendo la primera letra de cada una en mayúscula.

Por ejemplo, para "mi programa de suma" usaremos la notación "MiProgramaDeSuma". Esta sería la representación en "camel case" o "upper camel case" y lo vamos a utilizar para nombrar a los programas.

Para las variables, sin embargo, usaremos la variación "lower camel case", por lo que la primera letra será siempre minúscula. Entonces, en nuestro caso "nombre de la persona" será llamado "nombreDeLaPersona", empezando con minúscula.

Definidas las palabras reservadas, veremos cómo éstas se combinan entre sí para formar un programa.

Motivo: Para la mayoría de lenguajes de programación, si definimos una variable con espacios en blanco adentro, el intérprete/compilador no lo va a entender...

...por eso es que sacamos los espacios en blanco y ponemos todo junto:

Con respecto a la estructura, definiremos que todos nuestros programas deberán respetar el siguiente formato:

```
programa <<nombre del programa>>

inicio
    instrucción 1
    instrucción 2
    instrucción 3
    ...
    instrucción N
fin
```

Cuando programamos (ya sea con pseudocódigo o en un lenguaje de programación determinado) debemos tener en cuenta que cada instrucción va en una única línea de código y que las operaciones se deben escribir de la forma más clara que sea posible, para permitir que cualquier programador (o no) las entienda sin mayor esfuerzo. Es lo que se conoce como “claridad de código” y es una de las prácticas más importantes de la programación.

Hay que tener siempre en cuenta que una porción de código que creamos (o modificamos) nunca sabemos cuántas otras veces será visto y modificado por nosotros mismo o por otra persona. Por lo que generar un código legible siempre debe tomarse como una inversión, en lugar de una pérdida de tiempo.

De la estructura definida arriba, podemos rescatar un punto muy importante que hace a la claridad del código escrito: la indentación. Este término (se trata de un anglicismo) hace referencia a los espacios dejados entre el margen izquierdo en las líneas que contiene la “instrucción 1”, “instrucción 2”, etc. Este conjunto de espacios en blanco (también conocido como “sangría”, “tabulado” o “tabs”, haciendo referencia a esa tecla en el teclado de una PC), permite identificar los distintos “niveles” que tiene el código. Que las 4 instrucciones mostradas arriba tengan “1 tab” de corrimiento con respecto al margen, permite ver como éstas son agrupadas dentro de un bloque o nivel que comienza en “inicio” y finaliza en “fin”.

En la misma sintonía tenemos el siguiente punto planteado a continuación.

1.2 Documentación de código

La programación (como se dijo anteriormente) es sólo una parte dentro de un proyecto de desarrollo, complementado con otras actividades entre las cuales se encuentran la generación de la documentación del proyecto.

En relación a esta documentación, se encuentra un tipo de documento que surge directamente desde el programador, que está orientada a explicar y clarificar los aspectos más técnicos de la programación en el proyecto.

Esto es a lo que nos referimos con “documentación de código”: el hecho de agregar comentarios en forma narrativa, ad hoc, al código fuente que estamos programando con la finalidad de explicar un algoritmo complejo, aclarar qué modificaciones se fueron introduciendo, quién es el autor del programa, o cualquier dato relevante que sea considere imprescindible para comprender un bloque de código.

Para incluir esta clase de aclaraciones en el código, usaremos el operador “//” seguido del comentario, repitiendo la operación por cada línea a la cual se la quiera comentar.

Como mínimo y regla para este curso deberemos comentar (documentar) los siguientes puntos:

- Cabecera del programa:
 - Autor del programa
 - Fecha de creación
 - Descripción general del programa
- Cuerpo del programa
- Todas aquellas instrucciones complejas o que no resulten intuitivas

Un ejemplo podría ser:

```
//AUTOR: Emilio Rasic
//FECHA: 9/12/2018
//DESCRIPCION: este es un ejemplo de comentario general del programa
//que utilizaremos en el curso, pero dividido en dos líneas

programa NombreDelPrograma
inicio
    instruccion 1      //este es un tipo de comentario en línea
                        //que explica una determinada instrucción
    instruccion 2
    //este tipo de comentario se puede usar para explicar el bloque
    //de código que viene debajo
    instruccion 3
    instruccion N
fin
```

Ejemplo de comentario en el código:

programa limpiaAuto

inicio

sacaAfuera *// Saca el auto de la casa, hacia la vereda.*

lavaAuto *// Limpia el auto con agua y jabón.*

fin



ACTIVIDAD DE REFLEXIÓN:

¿Por qué es importante "documentar" el código? ¿Quién lo hace? Cuándo se hace:
¿durante la programación, antes o una vez que se finaliza? ¿Es realmente una ayuda?
¿A quién iría dirigida?

Ayudita importantísima sobre ingeniería de software:

*La calidad debe mantenerse DURANTE el desarrollo de un software,
es parte del día a día,
NO es un aderezo que se agrega de forma aislada AL FINAL del mismo.*

**Tómense un momento para pensarlo y pueden buscar información
online...**

UNA POSIBLE RESPUESTA:

Es fácil concluir que la documentación de código constituye una "buena práctica" de la programación, aunque también genera controversia entre los desarrolladores más experimentados quienes sostienen que un código desarrollado de manera inteligente (por ejemplo, usando nombres autodescriptivos) no necesita de comentarios, ya que esta necesidad reflejaría la creación de un código de mala calidad.

A pesar de esto, podemos decir que, de todas maneras, es importante comentar el código tanto para nosotros mismos como para aquellos desarrolladores con los que trabajamos en nuestro proyecto: aclaraciones sobre su diseño y propósito serán agradecidas para el siguiente programador que tenga que leerlo, editarlo y/o refactorizarlo, pudiendo hacerlo de una manera más eficiente y amena, ahorrando tiempo en su comprensión.

El mejor momento para realizar estos comentarios es durante el "tiempo de programación" para llevar un registro ordenado de lo que se va desarrollando o en caso de tener que interrumpir la actividad, y luego tener que retomarla, no "perder el hilo".

Seguimos...

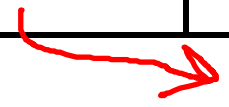
1.3 Operadores aritméticos y relacionales

El pseudocódigo permite la utilización de un conjunto de operadores aritméticos y relacionales que pueden ser utilizados para la construcción de las instrucciones necesarias.

A continuación, un detalle de los más usados:



	Operador	Significado	Ejemplo de uso
Relacionales	>	Mayor que	10>9
	<	Menor que	3<5
	=	Igual que	1=1
	<> o !=	Distinto de	1<>2 o 1!=2
	>=	Mayor o igual que	9>=7
	<=	Menor o igual que	3<=7
Aritméticos	+	Suma	6+3
	-	Resta	4-2
	*	Multiplicación	3*5
	/	División	10/4
	^	Potencia	4^2
	%	Resto de división (Módulo)	6 % 3



Se usa para "separar por casos", ¡está buenísimo! :3

La jerarquía de los operadores matemáticos es la misma a la del álgebra, aunque a través del uso del paréntesis puede modificarse.

1.4 Desarrollo en pseudocódigo

A continuación, desarrollaremos nuestro primer programa en pseudocódigo.

Comenzaremos por documentar (acción también conocida como “comentar”, referido al hecho de agregar comentarios documentales al código) el programa. Posteriormente recrearemos el algoritmo en el cuerpo del programa.

```
//AUTOR: Emilio Rasic
//FECHA: 16/01/2019
//DESCRIPCION: este programa muestra los 3 primeros números consecutivos a
//a partir de un número que es ingresado por el usuario

programa SerieDeTresNumeros
inicio
    ingresar: nroInicial          //se ingresa el primer número (*)
    //a continuación se realizan las sumas para
    // obtener los números de la serie (**)
    primerNro = nroInicial + 1
    segundoNro = nroInicial + 2
    tercerNro = nroInicial + 3
    //se muestran los 3 números de la serie (**)
    mostrar: primerNro
    mostrar: segundoNro
    mostrar: tercerNro
fin
```

No nos detendremos en la solución, sino que más bien repasaremos los elementos del programa para identificarlos claramente:

- El programa tiene un encabezado con un comentario inicial (autor, fecha y descripción del programa)
- El **programa** tiene un **nombre** descriptivo, escrito en “UpperCamelCase”
- El programa tiene un bloque principal, definido entre las palabras reservadas “inicio” y “fin”

- El contenido del bloque principal del programa está indentado, lo que permite ver la estructura del programa
- El contenido del bloque principal del programa tiene comentarios “en línea” y “por grupo de líneas”, refiriéndonos al primero como el que tiene un (*) y al segundo con los que tienen un (**)

1.5 Conclusiones

El pseudocódigo nos permite:

- Identificar inequívocamente cuál es el comienzo del programa
- Identificar claramente todos los puntos de finalización del programa
- Definir un número finito de instrucciones, estados intermedios y caminos entre el inicio y el final o las finalizaciones del programa
- Visualizar claramente los niveles de las estructuras del programa, a través de la indentación
- Comprensión de algoritmos por parte de personas que no tengan necesariamente conocimientos de lenguajes de programación
- Se encuentra basado en el lenguaje natural en el idioma de cada uno, por lo que **se evitan palabras complejas** o notaciones que no se interpreten inmediatamente
- Al ser un subconjunto del lenguaje natural, **se evitan errores gramaticales, abreviaciones y puntuaciones que podrían dificultar la comprensión.**

2. Ejemplo integrador

A continuación, desarrollaremos un ejemplo simple (desde el punto de vista del problema y de la estructura con la que serán resueltos – que probablemente no sea la óptima, aunque sí válida).

Justamente por la simplicidad de los ejemplos evitaremos **realizar el proceso de análisis de “Entrada-Proceso-Salida”**, aunque instamos a realizarlos **cada vez que sea posible y justificable.**

PROBLEMA: Dado un número inicial, mostrar los siguientes 3 números naturales inmediatamente sucesivos.

A continuación, veremos 3 posibles soluciones algorítmicas igualmente válidas para el problema planteado.

Esto nos deja entrever una de las tantas máximas que rigen el mundo de la programación:

“Para un problema dado, existen innumerables alternativas de solución”

Recomendamos analizar las 3 alternativas y posteriormente ver las explicaciones de cada una de ellas.

Versión 1:

```
programa MostrarNumerosV1
inicio
    ingresar: numero

    primerNro = numero + 1
    segundoNro = numero + 2
    tercerNro = numero + 3

    mostrar: primerNro
    mostrar: segundoNro
    mostrar: tercerNro
fin
```

En este caso, se ingresa un valor inicial, que se guarda en el elemento "numero". Este elemento (llamado "variable") se puede tomar figurativamente como una "bolsa" donde se ingresan distintos tipos de valores: en este caso, el número sobre el cual se debe construir la serie.

Posteriormente, se definen 3 nuevas variables (nuestras famosas "bolsas") donde se guarda el número inicialmente ingresado, al cual se le suman 1, 2 y 3, para obtener los siguientes valores que forman la serie.

En última instancia, se muestran los 3 valores y finaliza el algoritmo.

Un caso de prueba sencillo sería ingresar 2 valores (uno por cada vez que se ejecuta el algoritmo): primero 5 y luego 99, para tener un rango amplio de valores.

Repasemos el algoritmo para el primer caso de prueba:

1. Se ingresa 5, por lo que "numero" = 5
2. Luego, se le suma 1 a "numero" (que vale siempre 5) y se lo guarda en una variable llamada "primerNro". Por lo que "primerNro" = $5 + 1 = 6$
3. Se le suma 2 a "numero" y se lo guarda en otra variable llamada "segundoNro". Por lo que "segundoNro" = $5 + 2 = 7$
4. Se le suma 3 a "numero" y se lo guarda en una tercera variable. Por lo que "tercerNro" = $5 + 3 = 8$
5. Luego se muestran los valores de cada una de las variables:
 - a. primerNro = 6
 - b. segundoNro = 7
 - c. tercerNro = 8

Repetimos la ejecución del caso de prueba con otro valor a probar:

1. Se ingresa 99
2. A 99 se le suma 1 (primerNro=100)
3. A 99 se le suma 2 (segundoNro=101)
4. A 99 se le suma 3 (tercerNro=102)
5. Se muestra
 - a. 100
 - b. 101
 - c. 102

El algoritmo cumple correctamente con ambos casos de prueba, por lo que lo damos por satisfactorio.

Versión 2:

```
programa MostrarNumerosV2
inicio
    ingresar: numero

    primerNro = numero + 1
    mostrar: primerNro

    segundoNro = numero + 2
    mostrar: segundoNro

    tercerNro = numero + 3
    mostrar: tercerNro
fin
```

Pasemos a analizar la segunda alternativa de solución.

En este caso, la lógica del planteo es el mismo que en la alternativa de Versión 1.

La diferencia del planteo radica en el orden de los pasos de ejecución. En el primer caso, primero se realizaban todas las sumas y finalmente se mostraban todos los resultados, y en este caso a medida que se van calculando los números de la serie, se muestra cada uno de los resultados parciales.

El resultado final es idéntico en ambos, sumado a que el planteo también es similar.

Si "ejecutamos" los casos de prueba, comprobaremos que nuevamente se cumple con la premisa del problema planteado.

Versión 3:

```
programa MostrarNumerosV3
inicio
    ingresar: numero

    numero = numero + 1
    mostrar: numero

    numero = numero + 1
    mostrar: numero

    numero = numero + 1
    mostrar: numero
fin
```

Para finalizar con el presente ejemplo, pasamos a analizar la tercera versión de la solución.

Este algoritmo presenta una importante diferencia con respecto a los 2 primeros. Es similar al segundo, ya que a medida que va calculando los números de la serie, va mostrando los resultados.

La diferencia radica en el manejo del incremento del valor inicial ingresado. Vemos que no existen las variables intermedias ("primerNro", "segundoNro" y "tercerNro"), sino que todas las operaciones se realizan sobre "numero". Por esto es que no se suman los valores 1, 2 y 3, sino que es el mismo "numero" el que se va modificando (y mostrando).

En este algoritmo no podríamos mostrar los 3 resultados parciales al final, ya que estos estados intermedios se van "pisando" y perdiendo a medida que el algoritmo avanza.

Nos detenemos para validar el algoritmo, a través de la "ejecución" de un caso de prueba. En este caso haremos una única ejecución de prueba ingresando el valor 12.

1. Se ingresa 12, por lo que "numero" = 12
2. Luego, se le suma 1 a "numero" que pasa a valer 13
3. Se muestra este valor: 13
4. Se le suma 1 a "numero" que valía 13, por lo que pasa a valer 14
5. Se muestra este valor: 14
6. Se le suma 1 a "numero" que valía 14, por lo que pasa a valer 15
7. Se muestra este valor: 15

Vemos que los valores mostrados por el programa fueron 13, 14 y 15, por lo que damos el algoritmo por válido.

Es importante tener en cuenta lo visto en este ejemplo: para un caso problema sencillo, rápidamente identificamos 3 soluciones posibles válidas. Queda claro la multiplicidad de alternativas de solución, lo cual suele potenciarse a media que los problemas planteados aumentan en complejidad.

La pregunta en este caso podría ser: **¿cuál es la mejor solución?** Si bien existen ciertos parámetros que nos pueden guiar para seleccionar una de las tres como la más óptima, esta clase de juicios suele ir refinándose con el tiempo y la práctica. En complemento (claro está) del **buen juicio, sentido común y mejores prácticas** que direccionen una tipología de problemas en particular.

¿Qué nos conviene: la solución más fácil de entender, o la que menos tiempo y recursos utiliza...?

*A fines de comprensión, las dos primeras soluciones son más claras.
Sin embargo, la tercera solución usa 1 sola variable: ocupa menos memoria...*

*De todos modos, a nivel pseudocódigo, es importante **elegir la solución que más simple** nos parezca, la que mejor se entienda (ya que las consideraciones de eficiencia pueden evaluarse de otras maneras y también dependerán de la implementación).*

3. Tips sobre el desarrollo en pseudocódigo

- La indentación implica ^{la} correr el inicio de una línea de código hacia derecha (normalmente un "tab" o 4 espacios). Esto también se conoce como tabulado o sangría. **TODAS las líneas que se encuentran en un bloque deben estar indentadas.** El único bloque que tenemos de momento es "inicio" y "fin", por lo que todas las líneas (instrucciones) que estén adentro, debe empezar con el margen o sangría agregados.
- **Hasta el momento,** el universo de nuestro pseudocódigo se limita a las siguientes **palabras reservadas:**
 - programa
 - inicio
 - fin
 - ingresar:
 - mostrar:
- Además de esto, podemos hacer operaciones, como ya vimos. Si vamos a realizar un cálculo, es probable que lo hagamos para obtener un resultado. Ese resultado lo vamos a guardar en una variable. **[La asignación siempre se hace colocando la variable que guarda el resultado a la izquierda, luego el símbolo igual ("=") y luego el valor a asignar o la operación.]** Ejemplo:

valor a asignar = valor asignado

```
resultado = 100 + 200

valorA = 1000
valorB = 2000
resultado = valorA + valorB

resultado = valorA + 300

mensaje = "Este es un mensaje o valor alfanumérico por lo que se usa comillas dobles"

nombre = "Ramon Ismael"
```

Las 3 primeras asignaciones manejan valores numéricos. Las ~~otras~~ ^{últimas} 2 asignaciones usan valores alfanuméricos (letras, números y otros símbolos), por lo que esos valores deben ir entre comillas dobles ("").

- No tiene sentido hacer operaciones con variables que previamente no hayan recibido una asignación o un ingreso de valores. Ejemplo:

```
programa MostrarValores
inicio
    primero = 100
    segundo = "Hola mundo"
    ingresar: tercero

    //todos OK
    mostrar: primero      //valor numérico asignado
    mostrar: segundo      //valor alfanumérico asignado
    mostrar: tercero      //valor ingresado

    //incorrecto, "cuarto" no tiene valor
    X mostrar: cuarto
fin
```

- En las operaciones de asignación nunca se usa el dos puntos (":"), únicamente el igual ("=")
- Como ya vimos, para darle un valor a una variable, podemos optar por 2 caminos:
 - ingresar un valor: dando un valor a una variable usando la palabra reservada "ingresar:" como se ven en los ejemplos, simulando un valor desconocido que sería ingresado por el usuario del programa.
 - asignar un valor: darle un valor fijo a una variable

PERO • No tiene sentido ingresar y luego ^aasignar un valor una variable, porque se perdería el valor ingresado (es "pisado" por la asignación). Ejemplo:


```

programa PisandoValores
inicio
    //no tiene sentido
    ingresar: dato
    dato = "Antonio Vespucio"
    mostrar: dato    //se muestra Antonio Vespucio

    //no tiene sentido
    dato2 = "Nuñez"
    ingresar: dato2
    mostrar: dato2

    //tiene sentido
    dato3 = "La Máquina"
    mostrar: dato3

    //tiene sentido
    ingresar: dato4
    mostrar: dato4
fin

```

Como regla general, en la mayoría de las veces será un error ingresar y asignar (o viceversa) un valor a una variable en forma sucesiva.

- Cualquier instrucción que no pertenezca a las palabras reservadas antes mencionadas y a las operaciones permitidas, es INVÁLIDO.

Ejemplo:

```

programa IngresoDeDatos
inicio
    ingresar: nombreCancion
    mostrar: "La canción elegida es "
    mostrar: nombreCancion

    reproducir nombreCancion    //"reproducir" no es una instruccion válida
fin

```

- No se pueden combinar las operaciones de ingreso y asignación. Ejemplo:



ACTIVIDAD DE ANÁLISIS:

Escribir un programa en pseudocódigo que cumpla con los requerimientos planteados a continuación:

Requerimientos:

1. Se debe ingresar dos números, sumarlos y mostrar su resultado. Luego ingresar un tercer número, el cual se sumará al resultado de la suma anterior y mostrarlo el nuevo resultado.
2. Se debe solicitar ingresar 4 números y sacar el promedio de ellos.
3. Se debe solicitar el ingreso de cada día de la semana y se deben mostrar al final los datos ingresados.
4. Se deben mostrar dos números y luego sobrescribirlos con valores nuevos ingresados por el usuario. Mostrar los nuevos números ingresados.
5. Escribir un programa que descontará el stock de productos en un almacén. Se debe mostrar el stock inicial de productos disponibles, descontar la cantidad ingresada por el usuario de productos vendidos, y mostrar finalmente el número de stock actualizado.
6. Ingresar un valor que represente una dirección de correo electrónico, concatenarlo a un valor fijo que representa un mensaje y mostrarlos.

7. Crear un programa que permita ingresar el precio de un producto, luego el porcentaje de IVA y luego mostrar el que se paga de IVA y el precio del producto + IVA
8. Crear un programa que, a partir del saldo inicial de una cuenta (\$22.500), permita registrar un depósito de dinero en esa cuenta y que luego muestre el nuevo saldo final.
9. Crear un programa que permita ingresar los nombres de los países limítrofes al sur y al norte de Ecuador, además de la población de este país, y que luego ~~los~~ muestre los datos ingresados
10. Crear un programa que permita ingresar y mostrar el nombre de un alumno y sus notas en los 3 trimestres.
11. Crear un programa que permita calcula la hipotenusa de un triángulo en base a sus catetos
12. Crear un programa que permita calcula la edad de una persona en base al año de su nacimiento y el año actual (el año actual se debe ingresar por sistema)
13. Crear un programa que muestre, a partir del ingreso de un sueldo, el mismo sueldo, pero con un incremento del 20%
14. Crear una calculadora que convierta montos de dólares a pesos
15. Crear un programa que calcule y muestre el triple de un número ingresado por el usuario
16. Crear un programa para Claudia que le pida su apodo, su edad y le muestre los datos ingresados
17. Crear un programa que permita calcular el monto neto que le queda a una persona luego de descontarle a su sueldo el total de los gastos mensuales
18. Crear un programa que permita ingresar, calcular y mostrar el promedio de puntos de las últimas 3 temporadas de un club, junto con el nombre del club

19. Crear un programa que permita calcular un porcentaje cualquiera de un número cualquiera

20. Crear un programa que calcule el gasto total de llenar un tanque de combustible de un vehículo cualquiera

Condiciones de cumplimentación:

- El código debe estar **indentado**
- Usar **únicamente el pseudocódigo visto en el curso**

Podrán comparar sus respuestas con los ejemplos resueltos en el "ANEXO 1 - Respuesta Actividad de Análisis"

Previamente pueden consultar el "ANEXO 2 - Checklist de código".

IMPORTANTE:

- **¡Traten de resolverlo cada uno por sus propios medios!**
- **Las respuestas (el programa) NO debe ser enviado a los foros del Campus, salvo dudas puntuales de las respuestas.**

ANEXO 1 - Respuesta Actividad de Análisis

¿Trataron de resolver los requerimientos?

En caso afirmativo, avancen 3 páginas.

En caso negativo, ¡retrocedan 2 páginas y traten de resolverlos antes de ver las respuestas!

Página dejada intencionalmente en blanco

Página dejada intencionalmente en blanco

Página dejada intencionalmente en blanco

1)

```
programa SumaTres
inicio
    mostrar: "Ingrese un número"
    ingresar: nro1
    mostrar: "Ingrese otro número"
    ingresar: nro2
    nroTemp = nro1 + nro2
    mostrar: "Los primero dos números suman:"
    mostrar: nroTemp suma
    mostrar: "Ingrese el tercer número"
    ingresar: nro3
    nroTemp = nroTemp + nro3 suma + nro3
    mostrar: "Los tres números suman:"
    mostrar: nroTemp suma
fin
```

2)

```
Programa PromedioDeCuatro
inicio
    mostrar: "ingrese 1er. número"
    ingresar: nro1
    mostrar: "ingrese 2do. número"
    ingresar: nro2
    mostrar: "ingrese 3er. número"
    ingresar: nro3
    mostrar: "ingrese 4to. número"
    ingresar: nro4
    suma = nro1 + nro2 + nro3 + nro4
    promedio = suma / 4
    mostrar: "El promedio es: " + promedio
fin
```

3)

```
programa MostrarDiasDeLaSemana
inicio

    ingresar: nombreDiaUno //se ingresa el primer nombre
    //se almacena el nombre del día
    primerDia = nombreDiaUno
    ingresar: nombreDiaDos //se ingresa el segundo nombre
    //se almacena el nombre del día
    segundoDia = nombreDiaDos
    ingresar: nombreDiaTres //se ingresa el tercer nombre
    //se almacena el nombre del día
    tercerDia = nombreDiaTres
    ingresar: nombreDiaCuatro //se ingresa el cuarto nombre
    //se almacena el nombre del día
    cuartoDia = nombreDiaCuatro
    ingresar: nombreDiaCinco //se ingresa el quinto nombre
    //se almacena el nombre del día
    quintoDia = nombreDiaCinco
    ingresar: nombreDiaSeis //se ingresa el sexto nombre
    //se almacena el nombre del día
    sextoDia = nombreDiaSeis
    ingresar: nombreDiaSiete //se ingresa el séptimo nombre
    //se almacena el nombre del día
    septimoDia = nombreDiaSiete

    // se muestran los días ingresados
    mostrar: primerDia
    mostrar: segundoDia
    mostrar: tercerDia
    mostrar: cuartoDia
    mostrar: quintoDia
    mostrar: sextoDia
    mostrar: septimoDia

fin
```

4)

```
programa CambioDeNumeros
```

```
✓ inicio
```

```
    primerNumeroAlmacenado = 10  
    segundoNumeroAlmacenado = 20
```

```
    //mostramos los números almacenados  
    mostrar: "El primer número almacenado es"  
    mostrar: primerNumeroAlmacenado  
    mostrar: "El segundo número almacenado es"  
    mostrar: segundoNumeroAlmacenado
```

```
    ingresar: primerNumeroNuevo //se ingresa el primer número nuevo  
    //guardamos el primer número nuevo  
    primerNumeroAlmacenado = primerNumeroNuevo
```

```
    ingresar: segundoNumeroNuevo //se ingresa el segundo numero nuevo  
    //guardamos el segundo numero nuevo  
    segundoNumeroAlmacenado = segundoNumeroNuevo
```

```
    //mostramos el valor de los números almacenados  
    mostrar: "El primer número almacenado fue cambiado a:"  
    mostrar: primerNumeroAlmacenado
```

```
    mostrar: "El segundo número almacenado fue cambiado a:"  
    mostrar: segundoNumeroAlmacenado
```

```
fin
```

5)

```
programa ActualizacionDeStock
inicio
    stockActual = 100    //definimos el stock inicial
    //mostramos el stock actual
    mostrar: "El stock actual en el almacén es de:"
    mostrar: stockActual

    mostrar: "Ingresar cantidad de productos vendidos:"
    // se ingresa la cantidad de productos vendidos
    ingresar: cantidadProductosVendidos

    //restamos la cantidad de productos vendidos al stock inicial
    stockActual = stockActual - cantidadProductosVendidos

    //mostramos el stock actualizado
    mostrar: "El stock fue actualizado, el almacén cuenta con un stock de:"
    mostrar: stockActual
fin
```

6)

```
programa IngresaUnCorreo
inicio
    ingresar: correoElec    //Se guarda el correo electrónico ingresado
    mensaje = "Se enviara la respuesta a "
    mostrar: mensaje + correoElec
fin
```


7)

```
programa CalculoImporteMasIva
inicio
    mostrar: "Ingrese el precio neto:"          //solicita el ingreso del importe
    ingresar: precioNeto                        // asignamos precio neto a la variable
    mostrar: "Ingrese el % IVA:"                //solicita el ingreso % de IVA
    ingresar: porcentajeIva                     //asignamos %IVA a la variable
    iva = precioNeto * porcentajeIva / 100       //cálculo del IVA
    mostrar: "I.V.A.:" + iva                    //muestra el importe IVA
    importeTotal = precioNeto + iva             //cálculo del total
    mostrar: "Importe total del Producto:" + importeTotal //muestra el total calculado
fin
```

8)

```

programa DepositoCuenta
inicio
    totalCuenta = 22400 //indicamos la totalidad de dinero en la cuenta
    mostrar: "¿Qué cantidad desea depositar?:" //mostramos un texto y le pedimos al usuario que ingrese el (*)
    //monto a retirar el stock disponible (**)
    ingresar: valorDeposito //Se ingresa un valor numérico
    resultado = totalCuenta + valorDeposito //Se suma la variable TotalCuenta al valor ingresado (*)
    //ValorDeposito (**)
    //mostramos el estado de cuenta actual
    mostrar: "Su nuevo estado de cuenta es" + resultado
fin

```

9)

```
programa AprendiendoSobreEcuador
inicio
    mostrar: "País limitrofe al norte de Ecuador"
    ingresar: paisNorte
    mostrar: "País limitrofe al sur de Ecuador"
    ingresar: paisSur
    mostrar: "Población del Ecuador en millones de habitantes"
    ingresar: poblacion
    mostrar: "Ecuador es un país sudamericano que limita al norte con" + paisNorte +
        "y al sur con" + paisSur + ", donde viven un total de" + poblacion
fin
```

10)

```
programa NotasTrimestralesDeUnAlumno
inicio
    ingresar: alumno
    ingresar: notaPrimerTrimestre
    mostrar: "la nota de" + alumno + "es" + notaPrimerTrimestre
    ingresar: notaSegundoTrimestre
    mostrar: "la nota de" + alumno + "es" + notaSegundoTrimestre
    ingresar: notaTercerTrimestre
    mostrar: "la nota de" + alumno + "es" + notaTercerTrimestre
fin
```

11)

```
programa HallarHipotenusa
inicio
    mostrar: "ingresar primer cateto"
    ingresar: 1erCat
    mostrar: "ingresar segundo cateto"
    ingersar: 2doCat
    calculo = 1erCat^2 + 2doCat^2
    mostrar: "el valor de la hipotenusa es" + (calculo^0.5)
fin
```

12)

```
programa CalculoEdad
inicio
    mostrar: "Ingrese el año en que nació"
    ingresar: anioNac // Ingresar su año de nacimiento
    mostrar: "Ingrese el año actual"
    ingresar: anioAct // Ingresar año actual
    edad = anioAct - anioNac
    mostrar: "Ud cumplió o cumplirá " + edad + " años en " + anioAct
fin
```

13)

```
programa Aumento20PorcientoDeSueldo
inicio
    mostrar: "Ingrese su sueldo Actual" // se demanda la instrucción
    ingresar: sueldoActual // se carga la variable
    nuevoSueldo = sueldoActual * 1,2 // se realiza la operación en base a la variable
    mostrar: "Su nuevo sueldo es " + nuevoSueldo // se muestra el resultado de la operación
fin
```

14)

```
programa DolarAPesos
inicio
    mostrar: "Ingrese un monto en dólares"
    ingresar: montoUSD
    mostrar: "Ingrese el valor en pesos de 1 dólar"
    ingresar: valorUSD
    calculo = montoUSD * valorUSD //el valor asignado se multiplica por una cotización
    mostrar: "El valor en pesos es" + calculo
fin
```


15)

```
programa ObtenerElTripleDeUnNumero
inicio
    mostrar: "Ingresar un Número"
    ingresar: numero
    resultado = numero * 3
    mostrar: "El triple de " + numero + "es " + resultado
fin
```

16)

```
programa DatosPersonales
inicio
    nombre = "Claudia"
    mostrar: "Hola " + nombre + ", ingresa tu apodo"
    ingresar: apodo
    mostrar: "Bienvenida " + nombre + " de ahora en más serás " + apodo
    mostrar: "Indicanos tu edad"
    ingresar: edad
    mostrar: apodo + " tiene " + edad + " años"
fin
```

17)

```
programa CalculaEfectivoDelMes
inicio
    mostrar: "Calcule el efectivo que le va a quedar en el mes"
    ingresar: tuSueldo
    ingresar: cuentasQueHayQuePagar
    efectivoParaUsarEnElMes = tuSueldo - cuentasQueHayQuePagar
    mostrar: " Tu efectivo para usar en el mes es: " + efectivoParaUsarEnElMes
fin
```

18)

```
programa PromedioFutbol
inicio
    mostrar: "Ingrese el nombre del equipo"
    ingresar: equipo
    mostrar: "Ingrese los puntos de las ultimas 3 temporadas"
    ingresar: temporada1
    ingresar: temporada2
    ingresar: temporada3
    promedio = (temporada1 + temporada2 + temporada3) / 3
    mostrar: "El promedio de " + equipo + " es " + promedio
fin
```

19)

```
programa Porcentajes
inicio
    ingresar: numero
    ingresar: porcentaje
    resultado = numero * porcentaje / 100
    mostrar: "El " + porcentaje + "% de " + numero + " es " + resultado
fin
```

20)

```
programa Tanquelleno
inicio
    mostrar: "ingrese los litros del tanque a llenar"
    ingresar: litrosDelTanque
    mostrar: "ingrese el valor del combustible"
    ingresar: valorDelCombustible
    gasto = litrosDelTanque * valorDelCombustible
    mostrar: "para cargar el tanque se necesita" + gasto
fin
```

ANEXO 2 - Checklist de código

Algunas consideraciones a la hora de revisar su propio código:

Tema	Cumple	
	Si	No
Todas las asignaciones de valores a variables se hicieron usando el "="		
Todas las asignaciones de valores a variables se hicieron colocando la variable que recibe el valor a la izquierda del "=" y el valor nuevo o resultado a la derecha del "="		
Se incluyó el bloque "programa" y el nombre en UpperCamelCase		
Se incluyó el bloque "inicio" y "fin"		
Todo el código que está dentro del bloque "inicio/fin" está corrido hacia la derecha (está indentado)		
Todos los mensajes y texto alfanuméricos (letras y números combinados) están entre comillas dobles (")		
Ninguna de las variables está entre comillas		
Los "ingresar" y "mostrar" tienen ":" y no "="		
Ninguna variable con un valor asignado se está sobrescribiendo con un "ingresar:"		
El código está compuesto ÚNICAMENTE por las palabras reservadas (programa, inicio, fin, ingresar, mostrar), variables, valores y operaciones válidas		

Si todas las respuestas son afirmativas, hay muy altas chances que el programa esté libre de errores de código.

ANEXO 3 - Preguntas Frecuentes

1) P: Si utilizo Visual Studio Code para hacer los ejercicios, ¿es posible luego correrlos y ver si funcionan correctamente?

R: El pseudocódigo del curso no se puede ejecutar.

2) P: ¿Hay alguna plataforma para practicar pseudocódigo, como por ejemplo Pycharm o alguna otra?

R: La mejor plataforma para practicar el pseudocódigo del curso es con la cabeza de cada uno. Pensar lo que se hace, siempre. Entender, sobre todo.

Aplicaciones como Pseint no llegan a cumplir con nuestra misión: los terminan llevando hacia la prueba-y-error (que será más importante en el futuro que ahora) por sobre la comprensión lógica. Sumado a que no va a servir cuando veamos teoría de Objetos.



Igual te recomiendo siempre fijarte primero de antemano si funciona, por ejemplo sé que en el notepad de Windows el código no te toma el tab, solo los espacios de la barra espaciadora...

3) P: Para la indentación, ¿es lo mismo apretar la tecla de tabulación (tab) que poner 4 espacios con la barra espaciadora?

R: Si se está usando algún editor de texto (en los materiales del curso se usa Visual Studio Code), el valor de la tabulación ya viene por defecto con el equivalente a 4 espacios.

4) P: En uno de los programas resueltos se asigna el dato ingresado a otra variable:

ingresar: nombreDiaUno

primerDia = nombreDiaUno.

¿Por qué es necesaria la segunda línea y no se sigue trabajando con "nombreDiaUno"? *A veces te puede servir tener una copia de un valor (por ejemplo, para usar ese mismo valor en dos operaciones distintas)*

R: No es "necesaria", es solo un ejemplo de asignación y una muestra de diferentes opciones de resolución.

5) P: En el ejemplo sobre lo la documentación de código y hay distintas indentaciones para la "instrucción 1" y "2". ¿Por qué una aparece en la misma línea que la palabra "instrucción", y porque la otra figura en la línea de abajo?

```
//AUTOR: Emilio Rasic
//FECHA: 9/12/2018
//DESCRIPCION: este es un ejemplo de comentario general del programa
//que utilizaremos en el curso, pero dividido en dos líneas

programa NombreDelPrograma
inicio
    instruccion 1      //este es un tipo de comentario en línea
                       //que explica una determinada instrucción
    instruccion 2
    //este tipo de comentario se puede usar para explicar el bloque
    //de código que viene debajo
    instruccion 3
    instruccion N
fin
```

R: Son diferentes formas de **agregar comentarios: "en línea" o "de línea"**. Si es un comentario breve, puede ser "en línea". Si es un comentario más largo, mejor hacerlo "de línea". Lo más importante, en todo caso, es la prolijidad y, sobre todo, **evitar el scroll horizontal.**

 *No extenderse mucho horizontalmente (hace que sea más difícil seguir el hilo)*

6) P: Cuando se hace:

```
numero1 = 11
```

```
numero2 = 22
```

mostrar: "El resultado es " + numero1 + numero2

¿Se va a mostrar 33 o 1122?  "11" + "22" es "1122" (estoy juntando cadenas de texto)
11 + 22 es 33 (estoy sumando números)

R: Si se plantea así:

numero1 = "11"

numero2 = "22"

mostrar: "El resultado es " + numero1 + numero2

Se obtiene, literalmente:

"El resultado es 1122"

La diferencia está en las **""**. Este tema se va a ver con más detalle ^{en} la unidad 3 cuando veamos tipos de datos.

Los valores 11 y 22 con las **""** ~~se~~ indican ^{en} que son valores alfanuméricos, no numéricos. Por lo tanto, no se va a intentar hacer una suma aritmética sino una concatenación, como si fueran cualquier texto.

Teniendo:

numero1 = 11

numero2 = 22

mostrar: "El resultado es " + numero1 + numero2

Se obtiene, literalmente:

"El resultado es 33"

7) P: ¿No es recomendable usar la misma variable para guardar distintos resultados en un algoritmo? En lugar de usar "StockInicial" y "StockActual". Se usó solo "StockActual". ¿Cuál sería la regla para sí reutilizar la misma variable?

R: No hay reglas escritas para eso, pero sí podemos tener en cuenta algunas ideas al respecto. Por ejemplo: estaría mal (conceptualmente) guardar, por ejemplo, un nombre y un apellido en la misma variable, más aún si la variable no tiene un nombre descriptivo. ¿Se puede? Sí, claro, si se hace bien, se puede.

Dicho esto, reutilizar una misma variable tiene sentido cuando:

- Se va a ingresar o asignar en forma sucesiva un valor de una misma familia de datos (saldos, nombres, tamaños, capacidades, importes, etc.)
- Cuando, antes de reutilizar una variable, su valor actual ya fue utilizado y no se requiere.

Ejemplo erróneo:

mostrar: "ingrese el dato 1"

ingresar: dato

mostrar: "ingrese el dato 2"

ingresar: dato

mostrar: "ingrese el dato 3"

ingresar: dato

mostrar: dato

Esto es un error, porque se pisa sucesivamente el valor de "dato" y al final solo se muestra el último valor ingresado (los dos anteriores se pierden).

Mismo ejemplo, sin error:

mostrar: "ingrese el dato 1"

ingresar: dato

mostrar: dato

mostrar: "ingrese el dato 2"

ingresar: dato

mostrar: dato

mostrar: "ingrese el dato 3"

ingresar: dato

mostrar: dato

Modificación del ejemplo, más prolijo:

mostrar: "ingrese el dato 1"

ingresar: dato1

mostrar: "ingrese el dato 2"

ingresar: dato2

mostrar: "ingrese el dato 3"

ingresar: dato3

mostrar: dato1

mostrar: dato2

mostrar: dato3

8) *P: Sobre los nombres de las variables, sabiendo que deben ser autodescriptivas, la pregunta que surge es: ¿Afecta en algo al rendimiento del programa LA LONGITUD de los nombres de las variables?*

R: No, no hay impacto en el rendimiento, ya que siempre hay un límite de caracteres que puede tener el nombre de una variable (esto depende de cada lenguaje). Siempre va a ser más "pesado" el contenido de una variable que su nombre. Pero el contenido también va a estar limitado (por el "tipo de dato", temas que veremos en la unidad 3).

Por lo tanto, el problema no es el nombre sino los valores de las variables. Y claro, ahí sí que no es lo mismo tener dos variables que tener dos mil variables, claramente. Y no es lo mismo guardar en las variables un bit que un byte.

Si bien un nombre muy largo afecta claramente a la legibilidad del código, esto se mitiga (en parte) usando la forma lowerCamelCase y teniendo los nombres de las variables (y de los programas) claros.

¿De qué depende esto? Del sentido común, en gran parte. Por ejemplo, la variable para guardar el número de cuenta corriente en pesos de un cliente tipo "empresa" de un banco. ¿Qué nombre usaríamos?

¿nroCC?

¿nroCtaCteARSEmpresa?

¿nroCtaCtePesosEmpresa?

¿numeroCuentaCorrienteARSTipoEmpresa?

¿nroCCPE?

Entonces, ¿cuál de es la correcta?

Se puede pensar en "nroCtaCtePesosEmpresa", ya que claramente se puede inferir que la variable es de una cuenta corriente en pesos de empresa.

9) P: En los ejemplos a veces se dejan renglones en blanco entre las líneas del programa. ¿Hay una regla sobre el uso de renglones en blanco o cada uno los usa como le parece mejor para ordenar el código?

R: Es un estilo de programación, pero es preferible dividir las instrucciones en forma lógica y espaciada, en su justa medida. No es agradable ver código con espacio excesivos (nunca más de una línea en blanco entre instrucción e instrucción) ni tampoco el código apretado, sin espacios.

Viendo esto en programas simples, de 10 o 15 líneas, no parece tener mucho sentido, pero en programas de más de 100 líneas y diferentes "bloques" de código, todo esto tendrá mucho más valor.

Todo código tiene una complejidad intrínseca que es la lógica que implementa. Por lo tanto, el "deber" del programador es simplificar el resto de las complejidades todo lo posible, con espacios adecuados, comentarios de código relevantes y que aporten valor, indentando, etc.

10) P: ~~¿~~Los programas que hice no ^{son} ~~nos~~ exactamente iguales a las soluciones propuestas, ¿está mal? *Múltiples programas pueden satisfacer un mismo requerimiento.*

R: Si cumple con el requerimiento y está libre de errores (se cumple con el checklist, por ejemplo), se puede decir que no está mal. Pero es importante marcar que no existe una única forma de resolver un problema dado.

Por ejemplo: ¿cómo se hace para mostrar el número 2?

¿Así?

mostrar: "2"

¿O así?

dos = 2

mostrar: dos

O:

mostrar: 1 + 1

O:

mostrar: $(4/2) + 2 - 2$

¿Cuál está mal? Ninguno. ¿Cuál podría llegar a usarse? Todas las alternativas porque la solución depende del contexto. ¿Cuál es la mejor? Depende del requerimiento. Como se puede ver hay muchas (¿infinitas?) soluciones válidas para los problemas, incluso para los más sencillos. Muchas de estas soluciones no se parecen entre sí.

Dicho esto: la resolución ¿en qué se diferencia del original? ¿Qué hace el programa original a nivel de entradas y salidas? ¿Su versión hace lo mismo? Son iguales. ¿Hace algo en una secuencia diferente, pero hace lo mismo que el original? Está bien. ¿Las salidas son diferentes? Entonces es otro programa. ¿Eso lo hace que esté mal? No, para nada, simplemente es otro programa.

11) P: ¿Porque es importante documentar el código?

Es importante:

- Para que el propio desarrollador encuentre partes del programa que escribió tiempo atrás

- Para facilitar el testeo del programa
- Para el futuro mantenimiento del programa
- Para que sea más sencillo para otro programador modificar/mejorar/corregir el código

11.1) P: *¿Quién hace la documentación de código?*

R: El programador en primera instancia

11.2) P: *¿Cuándo se hace?*

R: Durante la creación del programa, para ir aclarando porciones del código que no están claras a simple vista. Después, durante el testeo, o para indicar puntos que podrían mejorarse con el tiempo.

11.3) P: *¿A quién va dirigida la documentación de código?*

R: Al mismo autor del programa, al tester, a los compañeros de desarrollo, a los futuros mantenedores del programa. En fin, a cualquiera que deba trabajar sobre esa porción de código.

12) P: *En cuanto al uso de las comillas: ¿Por qué una variable no va entre comillas después de la palabra reservada "ingresar:"? Y a la hora de usar "mostrar:": ¿Cuál es la diferencia entre usar comillas a la hora de escribir un elemento y no usar comillas?*

R: Las comillas son para almacenar o mostrar una cadena de caracteres alfanuméricos. En el caso del "ingresar:", lo que se ingrese será guardando dentro de la variable mencionada, por eso no lleva comillas. En el caso del "mostrar:" se está indicando que se va a mostrar un mensaje (si tiene "") o el valor de una variable, por lo cual se escribe el nombre de la misma, sin "".

Es una regla: **una variable NUNCA va entre comillas.** Ni en una asignación, ni en una salida ni en un ingreso de datos.

Veamos el porqué:

```
programa GOAT
inicio
    nombre = "lionel andrés".
    mostrar: "nombre" //(1)
    mostrar: nombre //(2)
fin
```

Las salidas del programa son:

```
nombre
lionel andrés
```

Porque en (1) se está mostrando el valor fijo "nombre" y en (2) el contenido de la variable "nombre".

13) P: *¿Cuál es la diferencia entre valor fijo y valor ingresado en una variable?*

R: Un valor fijo es un valor que le da el programador a una variable es forma explícita. Por el contrario, cuando se le da un valor a una variable que es desconocido por el programador, esto ocurre cuando un usuario ingresa un dato, usando la palabra reservada "ingresar:"

Por ejemplo:

edad = 41 //valor fijo definido por el **programador**

Versus:

ingresar: edad //valor desconocido ingresado por el **usuario**